A smaller checker envelope for iPRA

Inferred elimination premises and a finite-specialization obstruction

Research note prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We inspect the current `ConciseIPRA` proposals at commit `62396b06020f` (full hash recorded below). The published coordinatewise envelope of complete compact checkers is 256 significant Python tokens, 278 AST nodes, and 516 normalized bytes. Three delivered programs lower the corresponding coordinates to **253 tokens, 275 AST nodes, and 515 bytes**. The first two infer an elimination's major premise when its target instance matches the goal; the third uses the divisor pattern of the three meaningful formula arities. We give successful-trace and canonical-completeness arguments, exact controls, a readable strict checker, and reproducible executable evidence.

These are three different programs implementing the inherited arithmetic presentation. No single delivered program is below all three old minima, no mathematical axiom deletion is claimed, and the figures do not prove absolute minimality. The compact input contract remains explicit. Separately, we prove a mathematical obstruction: if existential witnesses are restricted to parameters and zero, specialization by any fixed finite menu of primitive recursive operations, even under arbitrary composition, does not recover the native consequences. A proper cut in a nonstandard model supplies a closed, guarded counterexample. This explains why two separately complete restrictions in the current corpus cannot simply be combined.

1 What improves, and what the benchmark means

The comparison is against the actual repository snapshot, not an older draft. Its README records ninety packages and measures complete Python modules after removing docstrings, applying `ast.unparse`, and appending one line feed, under CPython 3.12.14 [1]. The counted module includes its JSON reader, scope handling, proof checking, and the final `ACCEPT` output. The standard library and interpreter are common infrastructure for both old and new programs.

Complete program	Tokens	AST nodes	Bytes
Pinned token control	256	283	594
New token profile	**253**	280	599
Pinned AST control	260	278	602
New AST profile	257	**275**	607
Pinned byte control	265	306	516
New byte profile	264	304	**515**
Old coordinatewise envelope	256	278	516
New coordinatewise envelope	**253**	**275**	**515**

The token and AST controls are `kernel_refinement`'s selected profiles [2]; the byte control is the separate compact supplement of `canonical_recursors` [3]. That supplement still implements the old scalar-iterator calculus: its 516-byte program is not the checker for its successor-seeded recursor theory.

There is no proved global "lower bound" here. There are empirical minima among inspected implementations. The new envelope is strictly lower in each coordinate, by 3, 3, and 1, respectively. It is not the measurement of a fictional 253-token, 275-node, 515-byte single program. In particular, the token and AST profiles grow by five bytes against their own matched controls. If the request is interpreted as requiring one program strictly below all three old minima, this report does not achieve that stronger requirement. Nor does it provide a new smaller mathematical axiom basis. Its positive result is a reproducible improvement under the repository's coordinatewise comparison, together with the obstruction theorem below.

The compact programs assume a canonical closed guarded public goal. They are not exact validators of every possible claimed public formula. The strict readable companion checks the public grammar and rejects every input leaf except exact integers; its larger size is measured separately. We do not silently charge that additional validation as zero bytes.

2 The arithmetic and the complete logical interface

2.1 The target theory

The target is full first-order *intuitionistic* primitive recursive arithmetic: intuitionistic predicate logic with equality, symbols and defining equations for each finite primitive recursive description, successor injectivity and separation from zero, and induction for quantifier-free formulas, with passive parameters. It is not merely equational PRA, its negative translation, or the set of its true numerical equations.

We retain the scalar language already justified in the repository:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

Only u is bound in h in an iterator. The bound variables of D are distinct. All substitutions are capture avoiding. Write $1 = S0$ and

$$\mathrm{C}(v, a, b) = I(v, a; u.b)$$

with unused u. The ordinary meanings of the two binders are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

The designated native contradiction is $0 = 1$.

2.2 A complete native reference basis

For clarity, the following is the native reference calculus. No ambient classical logical rules are tacitly available. Contexts retain formulas; a displayed local assumption is discharged.

Primitive	Axiom or inference schema
Hypothesis	From $A \in \Gamma$, infer $\Gamma \vdash A$.
Zero computation	$\Gamma \vdash I(0, a; u.h) = a$.
Successor computation	$\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]$.
Transport	From $\Gamma \vdash s = t$ and $\Gamma \vdash F(s)$, infer $\Gamma \vdash F(t)$ for a native formula template F.
Closure use	From $\Gamma \vdash \mathsf{Q}_y B$ and $\Gamma \vdash B(0)$, infer $\Gamma \vdash B(t)$.
Paired opening	From $\Gamma \vdash \mathsf{D}_{x,y} B$ and $\Gamma, \mathsf{Q}_y B(b, y) \vdash A$, infer $\Gamma \vdash A$; b is fresh for the outer context, parameters, and A.
Case introduction	From $\Gamma, B(t, 0) \vdash B(t, Sz)$, infer $\Gamma \vdash \mathsf{D}_{x,y} B$; z is fresh for the outer context and t.
Equation progress	From $\Gamma, E(z) \vdash E(Sz)$, infer $\Gamma \vdash \mathsf{Q}_y E(y)$; E is a literal equation and z is fresh.

Reflexivity, symmetry, transitivity, congruence, explosion, numerical cases, quantifier-free induction, and the usual intuitionistic connectives are derived. For example, if $r = I(0, t; u.0)$, transporting the computation $r = t$ along itself with template $v = t$ gives $t = t$. The fact that a designated equation serves as contradiction is justified by derived explosion and the falsity bridge in Section 3; it is not a license to omit ordinary falsity from the target theory.

2.3 The smaller guarded presentation actually encoded

The compact checkers use an auxiliary calculus on the same terms and Eq/Q/D formulas. Its public formulas use Eq and D only, and every immediate equation matrix of a $\mathsf{D}_{x,y}$ must have the literal form

$$I(y, 0; u.h) = I(y, 0; u.k). \tag{G}$$

The bodies may depend on passive parameters and on the displayed variables. Auxiliary formulas inside proofs need not all satisfy the public guard. This public/auxiliary distinction is inherited, and is essential.

Replace the last two reference introduction schemes by the one root-determined scheme

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{Intro}$$

All eigenvariable conditions remain. Restrict the source of transport to terms other than the literal 0. Combine transport and closure use into the following single interface. Given a template F, a source s and a target t, the required major is

$$M(s, t, F) = \begin{cases} s = t, & s \neq 0 \text{ syntactically,} \\ \mathsf{Q}_v F(v), & s = 0 \text{ syntactically.} \end{cases}$$

The inference is

$$\frac{\Gamma \vdash M(s, t, F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{Use/transport}$$

Together with hypothesis, the two computation schemas, paired opening, and Intro, this specifies the six inference families being encoded. There is no unlisted induction principle for compound formulas.

At an atomic public matrix, the zero guard supplies its base, so progress suffices. Equations elsewhere can be padded by

$$L(s = t) = \mathsf{D}_{r,v}(\mathrm{C}(v, 0, s) = \mathrm{C}(v, 0, t)), \qquad r \text{ unused.}$$

One proves $L(E) \dashv\vdash E$ by introducing its successor row from E, and conversely opening it and using the closure at 1. Padding permits numerical cases on equations through compound case introduction, and then recovers equation induction. The source-zero restriction has an inherited admissibility proof, reproduced among the attributed foundation files. In particular, this restriction is not justified merely by saying that zero is "another variable."

For a completely guarded translation, put

$$p(s = t) = L(s = t), \quad p(\mathsf{Q}_y B) = \mathsf{D}_{r,y} p(B), \quad p(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} p(B),$$

with fresh unused r. The inherited guarded presentation theorem gives the same translated native consequences, with both formula round trips. The compact codecs use administrative zero-instance supply for computation; they are certificate implementations of this consequence relation, not additional arithmetic axiom schemas. The full decoder and its checking proof are included as attributed reference material [4].

3 Why the interpretation is a bi-interpretation

This section states the inherited bridge precisely enough to expose what the new checker changes depend on. The full arithmetic constructions are included under `reference/foundations/`; the new work does not claim to shorten those constructions or to reverify them in Lean.

Elaborate each fixed iterator body $h(u, \vec{p})$ into a staged first-order function symbol $i_h(n, a, \vec{p})$, with its two defining equations. There is no internal universal evaluator for arbitrary function codes. Let T_I use ordinary intuitionistic first-order logic with actual equality and genuine falsity, these equations, contextual induction for equations, and

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The last sentence is explicit: a theory of positive equations alone would admit a singleton structure, so it would not provide the claimed ordinary interpretation.

The arithmetic dependency order is as follows. From scalar iteration, equation induction, and numerical cases one constructs a scalar predecessor P with $P0 = 0$ and $P(Sn) = n$. Next come truncated subtraction and a characteristic of actual equality on the whole domain. These compile fixed quantifier-free formulas into characteristic equations and recover all quantifier-free induction instances. Pairing and projections can then be constructed and used to compile general primitive recursion. Pairing is not assumed in the predecessor proof.

For example, with a proved pairing π and projections L, H, the primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled using

$$\begin{aligned} V(n, \vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \\ f^*(n, \vec{p}) &= H(V(n, \vec{p})). \end{aligned}$$

Equation induction proves $L(V(n,\vec{p})) = n$ and the required defining equations. The other interpretation sends each fixed scalar iterator to ordinary primitive recursion ignoring the recursion-counter argument.

For the composites, internal equation induction proves uniqueness of two iterations with the same seed and extensionally equal updates. External induction on each finite description therefore proves that its double translation agrees with the original operation. Both numerical domains are the whole domain, defined by $x = x$, equality remains actual equality, and the comparison isomorphism in each theory has graph $x = y$. The ordinary meanings of D, Q and the derived logical macros supply the formula composites. Thus the bridge is a bi-interpretation, not an appeal to standard-model agreement or to equal proof-theoretic strength alone.

Proposition 3.1 (Inherited bridge preserved by the new programs)**.** *On the declared canonical guarded public language, each new checker accepts exactly the same theorem set as its matched control. Consequently its ordinary realization is bi-interpretable and equiconsistent with full first-order* iPRA*, with the same whole-domain identity composites.*

Proof. Sections 5 and 6 establish soundness and canonical completeness relative to the matched controls. Compose these proof translations with the inherited checking, guarded presentation, and arithmetic bridge theorems. No arithmetic interpretation is changed. □

4 The compact representation and its limits

Certificates are JSON arrays consumed from right to left. The last element is the goal; each remaining consumed instruction has four fields $[m, s, u, t]$. The fields are not a new numerical domain. They encode a finite proof and its term/formula templates. The meaning depends on the position, numerical or formula, in which an array is used.

Arity	Numerical position	Formula position
0	0	$\top$
1	successor	D
2	ignored annotation, meaning 0	equality
3	scalar iterator	Q (third child is the body)
≥ 4	ignored annotation, meaning 0	$\top$

Canonical closures are $[B, B, B]$; tripling a unary paired major produces its closure, whereas tripling another meaningful nonempty arity produces an administrative truth. Here $\top$ is the fixed equation $0 = 0$. Scalar formula padding also denotes $\top$. These are fixed semantic aliases, not arbitrary additional connectives.

The opener $i(a, t, d)$ adds $5 - \mathrm{len}(a)$ to its depth at every array. Meaningful increments are therefore $4, 3, 2$ for arities $1, 2, 3$. Numerical root depth is 3, formula root depth is 0, and protected numerical slots have width two. Under p protected slots, coordinates $3 + 2j$ and $4 + 2j$ name slot j for $0 \leq j < p$; the coordinates $0, 1, 2$ denote zero. Canonical terms choose the odd representative of each slot. Negative integers admitted by the floor k denote rigid free parameters. A numerical hole is filled only after its image is checked at numerical root depth 3. Missing images at used holes reject.

The scope floor decreases immediately before the continuation's goal and new hypothesis are opened. This ordering is retained. Although in a local profile $k = 1 - |\Gamma|$ on entry, it is false during that intermediate phase with the old context length. Replacing the state by context length without

repairing the phase would reject valid certificates and can spoil freshness reasoning. We do not make that change.

The compact acceptance protocol is successful process exit with exactly `ACCEPT` and one line feed. Exceptions reject, including under `python -O`; no trusted check is an `assert`. Unused certificate prefixes are inert. Public theorem claims are for the canonical guarded input contract. Neither the old nor the new compact profile is presented as an exact theorem recognizer for arbitrary strings, floats, Boolean leaves, or arbitrary public formula shapes.

5 Inferring the elimination major

The original packet supplied m even when the source, target, and template already determined it. Its relevant code was

```
if m == ([s, u] if s else [t] * 3) and i(t, u) == a:
    a = i(t, s)
```

The new token and AST profiles use

```
if i(t, u) == a:
    m = [s, u] if s else [t] * 3
    a = i(t, s)
```

The rest of the module still recursively checks the actual major m. Only its redundant supplied spelling is ignored on the matching branch. This is target-directed premise inference, not omission of a premise.

Theorem 5.1 (Successful-trace repair)**.** *For every successful new execution on the inherited compact interface, there is a repaired packet stream accepted by the old profile. Its goal is unchanged. Repair each consumed packet by replacing its first field with the value of m actually used after the new test.*

Proof. On a triggered instruction, the repaired first field is exactly $[s, u]$ when s is true and $[t, t, t]$ otherwise. The old major comparison and its target comparison therefore succeed, and both programs replace a by $i(t, s)$. On a non-triggered instruction the first field is left unchanged; the old conjunction cannot select an elimination because its target comparison is false whenever evaluated.

Both programs next check the same major, consume the same repaired substream for that proof, build the same retained hypothesis, and open the same continuation at the same fresh floor. Induct on the finite successful execution tree. The token profile's global floor is also identical: constructing the inferred major is pure and occurs before the recursive major proof. The local profile preserves the same tuple contexts. The repairs are determined by the successful new trace, not by a guess at which instruction was intended. □

Eager evaluation of $i(t, u)$ may newly reject malformed packets which the old conjunction short-circuited. This theorem does not assert equality of all raw acceptance languages. It does show that an accepted new proof cannot acquire an unproved arithmetic major.

Lemma 5.2 (No accidental elimination in canonical nonelimination packets)**.** *A canonical numerical tree has no binary array node, whereas every canonical Eq/Q/D formula contains a binary equality node. For numerical t, u, a successful formula-root opening $i(t, u)$ has no binary node. It therefore cannot equal a canonical formula goal.*

Proof. Canonical numerical arrays have arity one or three. Opening preserves those constructors and substitutes only numerical trees at numerical holes; scalar padding introduces no binary node. Structural induction gives the claim. Every finite canonical formula ends in equality. □

Theorem 5.3 (Canonical completeness of inferred majors)**.** *Every canonical derivation supported by the control has a certificate accepted by the new profile.*

Proof. Compile intended transport and closure-use instructions normally; their target tests hold and their inferred majors are the required premises. Computation and introduction packets have numerical t, u, so the lemma prevents unintended selection. At ordinary paired opening, set the inactive field t to 0; its target opening is scalar 0, which cannot equal a canonical formula. Hypothesis leaves need no instruction. The canonical compiler already establishes the required scope conditions. Induction on the derivation now completes the construction. □

Constructed elimination majors occur earlier in `uniform_zero`, and prevalidation occurs in `prevalidated_elimination` [1]. The measured change here is the target-match trigger and its placement in the two current smallest profiles. It is not a claim to have invented premise inference or removed an arithmetic axiom.

6 The one-byte arity refinement

The 516-byte control selects a fresh progress parameter using `~len(a) % 2`. Replace only that expression by `3 % len(a)`. The three meaningful formula arities have the table

n	1	2	3
$\sim n \bmod 2$	0	1	0
$3 \bmod n$	0	1	0

so the actual equality/case choice is unchanged. The new expression has one fewer significant token, two fewer AST nodes, and one fewer normalized byte.

Theorem 6.1 (Soundness and canonical completeness of the remainder selector)**.** *The 515-byte profile has the same canonical guarded public theorems as its 516-byte control, under the inherited decoder and input contract.*

Proof. On arities $1, 2, 3$ the decision is exactly the displayed table. At arity zero the new modulo operation raises an exception; this can only remove traces involving an empty truth alias. At arities at least four the formula is the fixed truth alias. Changing its zero instance to a fresh-parameter instance cannot create a new meaningful formula: opening preserves the outer arity and all children are ignored by the formula decoder. Introducing a paired truth or retaining a truth hypothesis is sound in either mode. For a computation-supply step the inherited proof uses its known zero instance and the freshness of the new parameters; it does not require the displayed goal's arity to confer a new induction rule. This same supply argument remains valid.

Thus the inherited successful-call soundness induction still applies. Canonical meaningful goals and introduction matrices have arity $1, 2, 3$, so their compiled proofs keep the old mode. Administrative truth exports can be retained without using an empty or surplus truth as a nontrivial pending goal. The canonical completeness compiler is unchanged. □

There is deliberately no claim of raw trace equality on empty and surplus aliases. In particular, applying a progress rule to a meaningful ternary closure would be wrong. Both selectors give zero at arity three.

7 A barrier to deleting both numerical arguments

The freedom to choose a witness and the freedom to choose a specialization target cannot, in general, be removed together. The result below concerns the consequences of a specified native calculus. It is not a lower bound on the size of every possible iPRA presentation, and it does not exclude a different interpretation with a different numerical domain.

7.1 The restricted calculus

Numerical terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where the last constructor binds only u in h. Its computation equations are

$$I(0, a; u.h) = a,$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Formulas are equations, $\mathsf{Q}_y B(y)$, and $\mathsf{D}_{x,y} B(x, y)$. Substitution is capture avoiding, and all variables displayed as fresh below satisfy the usual eigenvariable conditions.

Fix once and for all a finite family $F = (f_1, \ldots, f_m)$ of numerical operations, each specified by a fixed scalar term $f_i(x_1, \ldots, x_{r_i})$. The arities r_i are fixed and may be zero. For a numerical parameter scope V, let $\mathcal{T}_F(V)$ be the smallest family containing the parameters in V and all standard numerals, and closed under composition with the operations in F. This is a restriction on specialization targets; the general numerical term language still contains arbitrary iterator bodies. The family F is fixed for the whole calculus, not chosen anew for each proof.

Let $\mathcal{N}_F$ retain hypothesis, both computation equations, and the following rules of the native calculus:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash A(s)}{\Gamma \vdash A(t)} \qquad \text{(transport in any native formula template)},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad (t \in \mathcal{T}_F(V)),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A} \qquad (b \text{ fresh}),$$

$$\frac{\Gamma, B(w, 0) \vdash B(w, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \qquad (z \text{ fresh}, \quad w = 0 \text{ or } w \in V),$$

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)} \qquad (z \text{ fresh}, \quad E \text{ a literal equation}).$$

The fourth rule is introduction by numerical cases, and the fifth is equation progress. Contexts are lists of retained assumptions, so an assumption may be unused. Restricting the fourth rule further to the newest parameter, with 0 in an empty scope, gives a subsystem of $\mathcal{N}_F$.

No unrestricted structural rule substituting arbitrary numerical terms for the free parameters of a finished derivation is added to this restricted calculus, nor is its admissibility assumed. Such a rule would restore precisely a freedom removed by the restrictions. Capture-avoiding substitution within each displayed inference still has its usual meaning.

Theorem (finite-menu obstruction). *For every fixed finite family F, the calculus $\mathcal{N}_F$ fails to derive a closed, Q-free formula that the unrestricted native calculus derives. In particular, allowing a fixed finite menu of numerical operations as specialization targets does not repair the simultaneous restriction of introduction witnesses to 0 and parameters.*

The proof gives an explicit form of the missing formula. A direct majorization lemma first ensures that all functions used in its construction are scalar terms. Thus this argument does not need to assume the general compilation of every primitive recursive function into the scalar language.

7.2 A scalar majorant for a finite menu

In this subsection, inequalities and induction are used in the ambient theory PA, with definitional extensions by fixed primitive recursive functions. They are not additional rules of $\mathcal{N}_F$. Write

$$n + r := I(n, r; v.Sv).$$

PA proves the usual laws of this addition term.

Lemma (structural majorization). *For every fixed scalar term t, one can construct a unary scalar term $B_t(n)$ such that PA proves all three assertions:*

1. *$n < B_t(n)$;*
2. *$n < n' \implies B_t(n) < B_t(n')$;*
3. *if every free numerical argument of t is at most n, then $t \le B_t(n)$.*

Proof. Construct the terms by structural recursion on t:

$$\begin{aligned} B_0(n) &= Sn, & B_{x_j}(n) &= Sn, \\ B_{St}(n) &= S(B_t(n)), \\ B_{I(c,a;u.h)}(n) &= I\big(B_c(n), B_a(n); r.B_h(n + r)\big). \end{aligned}$$

When constructing B_h, regard the iterator variable u as one more free argument of h. The variable r in the last display is fresh. Each B_t has only its displayed parameter n free.

The zero, variable, and successor cases are immediate. For the iterator case, fix n and put

$$N = B_c(n), \qquad A = B_a(n), \qquad K_n(r) = B_h(n + r), \qquad W_j = I(j, A; r.K_n(r)).$$

The inductive properties of B_h give $K_n(r) > r$, so PA proves that W_j is strictly increasing in j and $W_j \ge A > n$.

Suppose the original free arguments are at most n. Their values in the original counter and seed satisfy $c \le N$ and $a \le A$. Compare the original iteration with W_j up to its actual counter c. If the original accumulator after j steps is at most W_j, then every argument of h, including that accumulator, is at most $n + W_j$. The inductive majorization property of B_h therefore bounds the next original accumulator by

$$B_h(n + W_j) = W_{j+1}.$$

PA induction on j proves that the original result is at most $W_c \le W_N = B_{I(c,a;u.h)}(n)$.

For strict monotonicity in n, take $n < n'$. Both the seed $B_a(n)$ and the counter $B_c(n)$ are strictly smaller than their counterparts at n'. The update $K_n(r) = B_h(n + r)$ is strictly increasing in r and increases when n increases. PA induction first compares the two iterations for the smaller number of steps, preserving the strict inequality between their accumulators. The additional steps of the

iteration at n' can only increase its value. This proves strict monotonicity of the displayed bound, as well as its inflationary property.

This is a metatheoretic induction on a fixed finite syntax tree. Every induction on numerical values used within it is a PA induction, so the resulting inequalities hold in every model of PA, including nonstandard ones. □

Apply the lemma to the finitely many terms specifying F, and define the unary scalar term

$$H(n) = n + 1 + \sum_{i=1}^{m} B_{f_i}(n).$$

The sum is a fixed finite term built from the addition macro; for $m = 0$ it is 0. PA proves that H is strictly increasing, that $H(n) > n$, and that

$$\bigwedge_{j=1}^{r_i} x_j \le n \quad \Longrightarrow \quad f_i(x_1, \ldots, x_{r_i}) < H(n). \tag{1}$$

Indeed, the majorization lemma bounds the left side's value by $B_{f_i}(n)$, and the additional summand $n + 1$ makes the final inequality strict. The same argument covers a nullary operation.

Define two more scalar terms:

$$J(k, n) = I(k, n; u.H(u)), \qquad g(n) = J(n, n).$$

Their defining equations give $J(0, n) = n$ and $J(Sk, n) = H(J(k, n))$. Since $H(x) > x$, PA proves

$$r < s \quad \Longrightarrow \quad J(r, n) < J(s, n). \tag{2}$$

There is no uniform evaluator for all primitive recursive programs hidden here: H, J, and g are three fixed finite scalar terms constructed from the fixed menu F.

7.3 An external cut closed under every permitted target

Work in the usual classical metatheory. By compactness, choose a nonstandard model $M \models \mathrm{PA}$ and an element $a \in M$ greater than every standard numeral. Interpret each fixed scalar term in M through its primitive recursive defining function. In particular, an iterator with a nonstandard counter is interpreted by its internal primitive recursive function in M, not by an external loop of nonstandard length.

Let $\overline{k}$ denote the numeral in M for a standard natural number k. Define the external subset

$$U = \{x \in M : \text{ for some } k \in \mathbb{N} \text{ in the metatheory, } M \models x < J(\overline{k}, a)\}. \tag{3}$$

The quantifier over k in this definition is an *external standard* quantifier. It is not a quantifier over all elements of M, and no definability of U within M is asserted.

The following properties follow from PA theorems already established. First, U is an initial segment of M. It contains all standard numerals, and it contains a, because $a < H(a) = J(1, a)$. Second, it is closed under successor: if $x < J(\overline{k}, a)$, then

$$Sx \le J(\overline{k}, a) < J(\overline{k+1}, a).$$

Every nonzero member y of U has a predecessor z in M; since $z < y$, that predecessor belongs to U. Consequently every member of U is either 0 or the successor of another member of U.

Third, U is closed under every operation in F. Given finitely many arguments $x_1, \ldots, x_{r_i} \in U$, choose one standard k large enough that all of them are less than $J(\overline{k}, a)$. Put $n = J(\overline{k}, a)$ in (1). Then

$$f_i(x_1, \ldots, x_{r_i}) < H(J(\overline{k}, a)) = J(\overline{k+1}, a).$$

Nullary operations satisfy the same bound without a list of arguments. It follows by induction on an ordinary finite target expression that every member of $\mathcal{T}_F(V)$ has value in U whenever the parameters in V have values in U.

Finally, $g(a)$ does not belong to U. For every standard k, $M \models \overline{k} < a$, and (2) gives

$$J(\overline{k}, a) < J(a, a) = g(a).$$

Thus $g(a)$ is below none of the bounds in (3). This proves that U is a proper cut, although it is closed under all the permitted specialization targets.

7.4 The two-carrier counterreading and rule soundness

Terms take values in the full carrier M. Numerical eigenparameters and the variables bound by Q and D range over U. For precision, define the value $\mathrm{val}_\rho(t) \in M$ and the truth value of a formula under *any* M-valued assignment ρ to its free variables. The numerical term interpretation is the one just specified. Write A_ρ for the truth of A under ρ, and write $B_\rho(c, b)$ for its truth after assigning $x = c$ and $y = b$; the one-variable notation is analogous. Formula truth is defined by

$$\begin{aligned}
(s = t)_\rho &\iff \mathrm{val}_\rho(s) = \mathrm{val}_\rho(t), \\
(\mathsf{Q}_y B(y))_\rho &\iff (B_\rho(0) \Longrightarrow \forall b \in U\, B_\rho(b)), \\
(\mathsf{D}_{x,y} B(x,y))_\rho &\iff \exists c \in U\, (B_\rho(c,0) \Longrightarrow \forall b \in U\, B_\rho(c,b)).
\end{aligned}$$

In these displays, substitution of a member of M means modifying the assignment, rather than inserting a new syntactic constant. A sequent is valid when it preserves truth for all assignments of its free numerical parameters to U.

This is a *two-carrier reading of the specified inference rules*. It is deliberately not an ordinary first-order structure for the complete term language on domain U: U is not closed under every term operation, as witnessed by $g(a)$. Nor is it the usual first-order reading on domain M, since the logical binders range over U. Soundness must therefore be proved rule by rule; it is not being inferred from ordinary first-order soundness.

The extension of the definition to arbitrary M-valued assignments gives the usual substitution lemma for numerical terms and for native formulas. In particular, replacing a term by a term of equal M-value preserves the truth of any native formula. The logical binders still range over the same fixed set U after such a replacement. The iterator's internal accumulator, in contrast, ranges over M through its numerical evaluation.

Lemma (soundness of $\mathcal{N}_F$). *Every derivable sequent of $\mathcal{N}_F$ is valid in the reading above.*

Proof. Induct on the derivation. Hypothesis is immediate, and both computation equations hold in M. Equality transport follows from the substitution lemma, regardless of whether the two equal term values lie in U.

For specialization, the induction hypotheses give the closure and its zero instance. The chosen target has value in U, by the closure property proved above. The required instance is therefore one of the instances asserted by that closure.

For elimination of D, its true major premise supplies some $c \in U$ for which the opened Q-formula is true. Assign the fresh parameter b the value c and use the induction hypothesis for the second premise. Freshness ensures that the truth values of the undischarged context and conclusion have not changed.

For introduction by cases, the chosen witness w has value $c \in U$, because it is 0 or an existing parameter. Fix an assignment making Γ and $B(c, 0)$ true. If $b \in U$ is 0, the desired instance is the assumed base. Otherwise $b = Sz$ for a $z \in U$. Assign that value to the fresh parameter of the premise. The induction hypothesis yields $B(c, Sz) = B(c, b)$. Thus the conditional closure holds at the permitted witness c.

It remains to check equation progress. Fix a parameter assignment making Γ true, and suppose $E(0)$ is true. The induction hypothesis for the premise says externally that

$$\text{for every } z \in U, \qquad E(z) \text{ true } \Longrightarrow E(Sz) \text{ true.}$$

For any fixed $b \in U$, every $z < b$ in M belongs to U, because U is an initial segment. As E is a literal equation, its truth is that of a fixed internal arithmetic formula of M, with its numerical parameters fixed. Hence

$$M \models E(0) \ \wedge \ \forall z < b\,(E(z) \longrightarrow E(Sz)).$$

PA proves the bounded induction consequence

$$(E(0) \wedge \forall z < b\,(E(z) \longrightarrow E(Sz))) \longrightarrow E(b). \tag{4}$$

Applying this theorem inside M yields $E(b)$. Since b was arbitrary in U, the required Q-closure is true.

No induction involving a predicate for U was used. The undischarged context may have external truth conditions involving U, but after fixing an assignment satisfying that context, the matrix in (4) is only the internal equation E. This is exactly where the literal-equation restriction on progress is used. □

The same argument handles the witnessed presentation of equation progress: first fix its permitted witness in U, then apply the last paragraph to the resulting equation. Imposing additional syntactic restrictions on transport, on specialization, or on the choice among the permitted witnesses cannot invalidate this soundness proof.

7.5 A closed native theorem that is lost

For numerical terms p, q define the selector

$$C(y, p, q) := I(y, p; u.q),$$

with u fresh, and define the native naming formula

$$\mathrm{Name}(t) := \mathsf{D}_{x,y}(C(y, t, x) = t),$$

where x, y do not occur free in t.

Lemma (the naming test). *Under any assignment in the counterreading,* $\mathrm{Name}(t)$ *is true if and only if* $\mathrm{val}_\rho(t) \in U$.

Proof. Write $p = \mathrm{val}_\rho(t)$. If the naming formula is true, choose its witness $c \in U$. Its zero instance is true because $C(0, p, c) = p$. Its instance at $1 \in U$ must therefore be true, and $C(1, p, c) = c$ gives $c = p$. Hence $p \in U$.

Conversely, if $p \in U$, select $c = p$. For every $y \in M$ one has $C(y, p, p) = p$: the zero case is the zero computation, and for $y = Sz$ the successor computation returns the constant body p. PA supplies this zero-or-successor distinction throughout M. In particular all the required instances for $y \in U$ are true, so the naming formula is true. □

Consider the closed Q-free formula

$$\Psi_g := \mathsf{D}_{r,v}\, \mathrm{Name}(g(v)), \tag{5}$$

where r is unused. Its base instance is true: $g(0) = J(0, 0) = 0$, and $0 \in U$. Its instance at $v = a$ is false, because $g(a) \notin U$. The witness r cannot change either fact. Thus Ψ_g is false in the counterreading and, by the soundness lemma, is not derivable in $\mathcal{N}_F$.

The unrestricted native calculus has a direct proof of Ψ_g. For any term t and a parameter w fresh for t, successor computation proves

$$C(Sw, t, t) = t.$$

Introduction by cases with the witness t therefore proves $\mathrm{Name}(t)$; the base assumption of that introduction is unused. In particular it proves $\mathrm{Name}(g(Sz))$ for a fresh z. Apply introduction by cases once more, now to the matrix $\mathrm{Name}(g(v))$ with unused existential variable r and dummy witness 0. This gives (5). Neither universal generalization nor an arithmetic completeness theorem is needed for this native derivation. The finite-menu obstruction follows.

A guarded public version. The failure can also be placed in the guarded public grammar. For an equation E of the form $s = t$, put

$$\mathrm{Gate}(E) := \mathsf{D}_{p,q}\big(C(q, 0, s) = C(q, 0, t)\big),$$

where p is unused and both bound variables are fresh. This formula has the same counterreading truth value as E. Its zero row is $0 = 0$; its row at $1 \in U$ is $s = t$; and if $s = t$, all its rows are true. The unrestricted native calculus derives $\mathrm{Gate}(E)$ from E by computation, transport, and introduction by cases.

Consequently the closed formula

$$\mathsf{D}_{r,v}\mathsf{D}_{x,y}\, \mathrm{Gate}\big(C(y, g(v), x) = g(v)\big)$$

is also false in the counterreading and directly provable in the unrestricted calculus: gate the computed successor equation in the naming proof just given. Its innermost matrix is literally of the guarded form $I(q, 0; u.h) = I(q, 0; u.k)$. This avoids attributing the obstruction solely to an unguarded auxiliary goal.

7.6 Consequences and limits

There is an elementary counterreading for the further restriction in which targets are only parameters and 1, with other numerals disallowed as targets. Take $M = \mathbb{N}$ and $U = \{0, 1\}$. The same rule-soundness proof works: cases on U use only 0 and $S0$, and equation progress over U uses only the step from 0 to 1. Then $\mathrm{Name}(S(S0))$ is already a false closed native theorem. This finite reading does not allow successor-of-parameter targets, since U is not successor closed. The nonstandard cut construction above is what handles those targets, and much larger fixed menus, correctly.

The theorem applies to every finite menu of scalar numerical terms. In particular it applies to fixed addition, multiplication, exponentiation, and any other represented primitive recursive operations. Using the established term-completeness of the scalar language, it can equivalently be stated for an arbitrary finite menu of fixed primitive recursive operations. An arbitrary supplied iterator body is different: it gives an unbounded family of target operations and is not covered by a fixed finite menu.

This obstruction is compatible with separate completeness results for restricted introduction witnesses and for restricted specialization targets. An unrestricted witness may leave U, making its introduction rule unsound in the present reading. An unrestricted specialization target may also leave U, making its elimination rule unsound. The counterreading therefore applies precisely when the two sources of freedom are restricted together.

Finally, the conclusion is a strict failure of the original native consequence relation, including on closed guarded formulas. Because the reading is not an ordinary first-order structure on U, it must not be repackaged as a theorem that every changed-domain interpretation is impossible. Such an interpretation would require its own translations, soundness proofs, and definable composite isomorphisms. None is ruled out by the statement proved here.

8 Executable evidence and reproducibility

The archive contains the three complete compact profiles, exact pinned controls, a readable strict companion, actual positive and negative certificates, source provenance, measurement code, and validation code. Run

```
python3 reproduce.py
python3 build.py
```

from the archive's top directory. The first command uses only Python's standard library. The second requires pdfLaTeX and the packages named in this source. CPython 3.12.14 reproduces the exact source measurements; other Python versions can have different `ast.unparse` output.

The inference validation compares the local and global scope profiles against their own controls. It exercises inherited valid certificates, future-witness capture and context-leak attacks, and deliberately erased major fields. Repaired control certificates are actual files, not just descriptions of a proposed repair. Normal and optimized Python runs are both checked. The byte validation covers the meaningful arities and the intentional truth-alias differences. Exact counts, outputs, and hashes are in the generated evidence files.

The final reproduction ran 631 separate checker processes without an unexpected outcome: 423 for inferred-major validation, 152 for the byte selector, and 56 for the strict companion. The inference suite includes 31 generated certificates with 7,607 redundant major fields erased and 31 actual repaired control certificates. The readable strict companion measures 580 tokens, 666 AST nodes, and 2,473 normalized bytes; it is an audit interface, not a new strict-checker size record.

These tests are finite executable evidence. They do not prove universal soundness, the nonstandard-cut theorem, or bi-interpretability. Those claims rest on the written arguments and the explicitly inherited bridge. No new Lean or Rocq verification is claimed. The repository's Lean development concerns the mathematical calculus and its translations; it does not automatically verify the delivered Python implementations. Python recursion and memory limits can reject very large valid certificates; the theoretical checking algorithm is understood with unbounded finite resources, as are the matched controls.

9 Further research questions

1. Can one complete program, with the same input and output contract, beat all three old minima simultaneously? The three separate new profiles do not answer this question.
2. Can a mathematical reduction lower the best complete strict checker, including public-language validation, instead of refining the compact codec? Any comparison should count normalization, typing, and decoding that move into the trusted front end.
3. Can a structurally generated infinite family of specialization targets be simpler than arbitrary terms while defeating the finite-menu obstruction? The theorem does not cover an iterator constructor whose body is supplied freely.
4. Does a useful quotient or different numerical domain evade the counterreading while retaining both interpretation composites? Failure of the identity/native consequence translation alone does not settle that question.
5. Can the semantic decoder and successful-call soundness theorem for the compact checker be formalized once, with verified small refinements as corollaries? This would give a stronger assurance boundary than testing each tiny source independently.
6. Can the arithmetic bridge itself be shortened, particularly the scalar predecessor and formula selector constructions, while retaining the full intuitionistic formula round trips? Reducing those mathematical dependencies would be more substantial than another byte reduction.

A The complete selected programs

The following are the entire measured modules, not extracted kernels. The line wrapping in this PDF is typographical; the files in the archive are the executable sources.

A.1 Token profile: 253 tokens

```
from json import load
k = 0

def i(a, t=0j, d=0):
    try:
        return [i(b, t, d + 5 - len(a)) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    while i(a) not in g:
        m, s, u, t = q()
        if i(t, u) == a:
            m = [s, u] if s else [t] * 3
            a = i(t, s)
        h = c(m, g) or m * 3 if m else [[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u
    else (a := i(*a, t, 2))
        k -= 1
        g += (i(h, (m or len(a) == 2) and k),)
        a = i(a, [k])
q = load(open(0)).pop
c(q(), ())
print('ACCEPT')
```

A.2 AST profile: 275 nodes

```
from json import load

def c(a, g=(0,), k=0):

    def i(a, t=None, d=0):
        try:
            return [i(b, t, d + 5 - len(a)) for b in a]
        except:
            return a if k <= a < d else d % 2 and i(t, d=3)
    while i(a) not in g:
        m, s, u, t = q()
        if i(t, u) == a:
            m = [s, u] if s else [t] * 3
            a = i(t, s)
        h = c(m, g, k) or m * 3 or ([[s and [s], u, t], i(t, [s, u, t], 3) if s else u] if u else
     (a := i(*a, t, 2)))
        k -= 1
        g += (i(h, (m or len(a) == 2) and k),)
        a = i(a, [k])
c((q := load(open(0)).pop)())
print('ACCEPT')
```

A.3 Byte profile: 515 bytes

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a <
     d else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = (m, a) == (s and [s, u] or [t] * 3, i(t, u)) and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a
    := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or 3 % len(a)) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

B Provenance and interpretation obligations

All frozen sources are pinned to `62396b06020f794fa6dc01b4025a6a5db9d4d28a`. `PROVENANCE.json` records full paths, original Git blob hashes, SHA-256 hashes, lengths, and permanent URLs. The original permissive license is retained. Foundation files are reproduced as reference material with their original notation and internal cross-references; they are not presented as newly authored lemmas or silently concatenated into this report.

The logical dependence is:

new successful-checking and canonical-completeness arguments

$\Downarrow$

pinned codec soundness and guarded presentation

$\Downarrow$

native scalar calculus and ordinary realization T_I

$\Downarrow$

whole-domain bi-interpretation with full first-order iPRA.

The finite-menu counterreading is a separate theorem about the displayed restricted native rules. It is not used to establish the new checkers' soundness, and is not a proof that all alternative axiomatizations must have the same complexity.

References

[1] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, README at commit `62396b06020f794fa6dc01b4025a6a5db9d4d28a`, 12 November 2026. Permanent repository source.

[2] ProveIt, `ConciseIPRA/Research/kernel_refinement`, especially the selected complete programs, claims, measured evidence, and attributed foundation sources, same pinned commit. Permanent package source.

[3] ProveIt, the compact supplement of `canonical_recursors`, especially `checker_compact_zero.py` and `ARGUMENT.txt`, same pinned commit. Permanent compact supplement.

[4] ProveIt, *A uniform arity encoding and its checking theorem*, `inherited_arity_checker.tex`, reproduced under `reference/foundations/`. Same pinned commit.

[5] ProveIt, *Arithmetic strength and both ordinary interpretation composites*, `inherited_ordinary_bridge.tex`, and the predecessor, equality-reflection, primitive-recursion, guarded-core, and zero-source-admissibility proofs, reproduced under `reference/foundations/`. Same pinned commit.

[6] ProveIt, the `newest_parameter` and `deterministic_specialization` packages, same pinned commit. These establish separate restrictions; the finite-menu theorem in this report examines their simultaneous restriction.